

FCMトークン管理設計に関するディスカッション記録

作成日時 (JST)	2026-06-04 12:16
テーマ	複数端末環境におけるFCMトークンの永続的な識別・更新ライフサイクル設計
対象アーキテクチャ	Web型SPA / Firebase Cloud Messaging (FCM) / Supabase Edge Functions

1. 決定された設計方針（サマリー）

複数端末やブラウザプロファイルが同一アカウント（ユーザーID）を共有する環境において、個々のクライアントを一意に識別し、FCMトークンの更新（ローテーション）を確実に追跡するため、以下の「**クライアント発行UUID方式**」を採用する。

- **永続識別キー（UUID）の生成:** ユーザーがログインを完了したタイミングで、アプリケーション側でUUID（v4等）を発行し、ブラウザの localStorage に永続化する。次回以降のアクセスではこの値を使い回す。
- **トークン管理のキー構造:** Supabaseのトークン管理テーブルでは、この UUID を主キー（またはユニークなインデックスキー）として扱い、FCMトークンと紐づける。これにより、デバイス情報やブラウザプロファイルを細かく解析・管理する煩雑さを排除する。
- **トークン更新処理:** トークンがリフレッシュされた際は、UUID と「新トークン」を Supabase Edge Functionsに送信し、該当するUUIDレコードのトークン列を書き換える。

2. Edge Functions 呼び出し（契機）の整理

FCMトークンをSupabaseへ登録・更新するために、Edge Functionsをコールするトリガー（契機）は主に以下の通り。

1. **アプリケーション初期化時（起動契機）:** アプリ立ち上げ時にFirebase SDKが初期化され、最新のトークンを取得したタイミング。初回登録、または未検知の変更を反映する。
2. **SDKによるトークン更新検知時（イベント契機）:** アプリ稼働中に、SDK側のコールバック（イベントリスナー）がトークンのリフレッシュを検知したタイミング。
3. **FCMサーバーエラー検知時（配信エラー契機）:** バックエンド（Edge Functions等）から通知を送信した際、FCMから「トークン無効（Bad Registration / Not Found）」のエラーレスポンスが返ってきたタイミング。この時はデータベースから該当レコードを削除する。

3. ディスカッション・タイムライン

ユーザー

通知を送った時にそのトークンが無効であるというレスポンスがもらえるんですね。

アシスタント

はい、その通りです。FCMに通知リクエストを送った際、トークンが無効である場合はエラーコードが返ってきます。

このエラーを検知したら、Supabaseのエッジファンクション内で該当するトークンをデータベースから削除する処理を組み込んでおくと、不要な送信を防いで常にリストをフレッシュに保つことができますよ。

ユーザー

例えば私が3つのパソコンで通知を仕掛けてたとして1つのパソコンのトークが無効になったら無効になるってFCMが渡って分かんないですか？パソコンが初期化されちゃうと。

アシスタント

ご指摘の通り、パソコンが初期化されたことを、FCMが自動的に検知することはありません。

FCMがトークンが無効だと判断するのは、実際にそのトークン宛てに通知を送信しようとした時です。

その送信の試みの際に、FCMサーバーからエラーが返ってきますので、そのエラーをキャッチした時点で、リストから削除するという運用にするのが一般的ですよ。

ユーザー

いや、だって一時的に電源が切れてるために端末まで通知が届いてないだけかもしれないじゃないですか。そうするとFCMサーバーはそのトークンが無効だっていうことをどうやって判断するんですか？

アシスタント

FCMサーバーは、一時的なオフライン状態と、アプリのアンインストールやデータ削除による無効化を区別して処理しています。

もし端末の電源が切れているだけの場合、通知はサーバー側で一時的に保持され、再びオンラインになった時点で配信が試みられます。

トークン自体が無効になるのは、アプリが削除されたり、OSレベルでトークンが更新されたりして、FCMのサーバーが配信先に到達できなくなった時です。

その時点でエラーコードが返ってくるので、それを検知してデータベースから削除するというのが、適切な対応となりますよ。

ユーザー

「長期間電波の届かない範囲にいる」ということと「アプリがアンインストールされた」ということの違いは何ですか？

アシスタント

長期間電波が届かない範囲にいる場合、端末はまだサーバーに登録された状態なので、通知は送信されてサーバーに保留されます。その後、電波が回復してオンラインに戻った時点で、配信が試みられます。

一方で、アプリがアンインストールされた場合は、OSのレベルでプッシュ通知の登録が無効になります。そのため、FCMサーバーは配信先に到達できないと判断し、エラーを返す仕組みになっています。

ユーザー

アプリがアンインストールされた場合、FCMサーバーから、えっと、その端末のOSとの通信のやり取りの中で、そのアプリはもうないよっていうような会話がされるんね。

アシスタント

はい、基本的にはそのような認識で合っています。

アプリがアンインストールされるとOSはそれを認識し、そのアプリ宛てのプッシュ通知を受け取れない状態になります。

そのため、FCMサーバーから通知を送ろうとしても到達できず、エラーが返ってくるため、それを検知してリストから削除するという仕組みになっていますよ。

ユーザー

アプリのインストールもしていない、OSも、えーと、特に、いー、変更していない、えー、で、電波も届く状態にある、という状態においても、なんかOSが、なんかさっきリフレッシュするだかなんだかで、えー、IDが変わるみたいなこと言ってませんでした？そういうことがあるんですか？

アシスタント

はい、その認識で間違いありません。

アプリ側のSDKが古いトークンを定期的に更新したり、Google側でセキュリティ上の理由でトークンを再発行したりすることがあります。

そのため、ユーザーが何も操作していない場合でも、トークンが変わってしまうことは十分に考えられますよ。

その変化に対応するためには、トークンが新しくなったことを検知し、データベースの値を自動的に書き換える仕組みを実装しておくのが大切です。

ユーザー

トークンが新しくなったことを検知するっていうのは、えーと、アプリ側でどうやって実装すればいいんですか？

アシスタント

アプリ側でトークンの更新を検知するには、FirebaseのSDKが提供するイベントリスナーを使用します。
例えば、ウェブアプリケーションであれば、メッセージングに提供されるトークン更新コールバックを設定します。

このファンクションの中で新しいトークンを取得し、Supabaseのデータベースに保存する処理を呼び出すことで、トークンの変更に対応することができますよ。

ユーザー

ウェブアプリケーションなんてずっと全面にいるわけではないじゃないですか。なので、アプリが動いてない時にトークンが更新されたら、それはどうやってトークン更新検知すればいいんですか？

アシスタント

アプリがアクティブでない間にトークンが更新された場合、その時点で直ちに検知することはできません。
そのため、ユーザーが次にアプリを開いたタイミングで、SDKが最新のトークンを取得し、Supabaseのデータベースを更新する処理を実行させるのが一般的です。

あるいは、通知を送ろうとした際に、FCMから無効であるというエラーが返ってきた時点でデータベースから削除するという方法も安全ですよ。

ユーザー

そうだね。アプリケーション立ち上げてすぐにFCMのフレームワークはトークンが変わったよってイベントを発火してくれるってことですか？

アシスタント

はい、その通りです。アプリケーションを立ち上げた際に、FirebaseのSDKが初期化される過程で、トークンが新しくなっているかどうかを確認されます。

もしトークンが変更されていれば、トークン更新のイベントが発火され、新しいトークンを取得できますので、そのタイミングでSupabaseのデータベースを更新することが可能ですよ。

ユーザー

そうすると更新前と更新後のIDをトークンを、ええ、Webアプリ側で認識してないと、このトークンがこのトークンになったよみたいな更新ができないってことじゃないですか？

アシスタント

その通りです。古いトークンと新しいトークンの両方を認識していないと、どれをどれに更新すればいいか特定できません。

通常は、新しいトークンを取得した際に、現在のユーザーIDなどと一緒にSupabaseのEdge Functionsに送信し、データベース上で紐づくレコードを更新する構造にします。

ユーザー

じゃ、古いトークンを、えーと、Web アプリケーション側も、えーと、ローカルストレージなどで保存しておかなきゃいけないってことですか？

アシスタント

いいえ、Webアプリケーション側で古いトークンを保存しておく必要は必ずしもありません。FirebaseのSDKがトークンの更新を検知した際には、直接新しいトークンを取得できますので、そのタイミングでユーザーの認証情報と一緒にSupabaseのエッジファンクションに送信し、データベースのレコードを最新のものに更新する処理を行うのが一般的ですよ。

ユーザー

私が複数のスマホを扱ってる場合、どのスマホのトークが更新されたのかは、アカウントIDからだけじゃ、えーと、判断できませんよね？

アシスタント

その通りです!アカウント IDだけでは、複数のスマートフォンのうち、どのデバイスのトークンが更新されたか判別できません。

FCMのトークンは、アプリのインストールごとに固有のものが発行されるため、正確に識別するには、デバイス固有のIDやブラウザ情報を一緒にデータベースに保存しておく必要があります。

ユーザー

オリジンによって変わったり、ログインするユーザーによって変わったり、あるいは、えーと、プロフィールによって変わったりっていうことを言ってました。だからそんなことを管理する、面倒なことを考えるよりもえーと、今まで持ってたトークンと、えー、新しくなったトークンっていうのが比較できればいいわけですよ。

アシスタント

その通りです!過去に保存したトークンを検索キーにして、新しいトークンに書き換えるというアプローチは、非常に合理的で優れた解決策です。

この方法であれば、デバイスやブラウザの情報を細かく追跡する必要なく、トークンの更新を確実に管理できますよ。

ユーザー

そのためにローカルストレージに保存しておかなければいけないですよって言ったんです、さっき。

アシスタント

申し訳ありません。おっしゃる通り、以前のトークンをローカルストレージに保存しておけば、新しいトークンと比較して変更を検知しやすくなりますね。

その場合、古いトークンを取得して新しいものと比べ、異なる場合にはデータベースの情報を更新するという流れになります。

ユーザー

また古いトークンを保持していると、今度その保存に関するトラブルが起きた時にもうあの、更新のチェーンが繋がなくなるっていうことになっちゃいますねと。だから、それに代わる何かそのアプリケーションとユーザーとデバイスとプロフィール、それら一切含めて全てをユニークにちゃんと管理するためのキーって、ナチュラルキーって紡ぎ出せるもんですかね？

アシスタント

そのような、アプリケーション、ユーザー、デバイス、プロフィールのすべてをカバーするユニークなキーを生成することは、考え方によっては可能です。

一般的には、これらの要素を組み合わせた複合キー、例えば「オリジン_ユーザーID_デバイス識別子」といったキーを構成する方法が考えられます。

ただし、デバイスやブラウザの識別を確実に保つには、プライバシー規制やユーザーの設定に影響を受ける場合もあります。

そうした課題を考慮すると、もっとシンプルに、データベースの認証システムを基盤としてセッションを管理するほうが、より安定して運用できる場合もありますよ。

ユーザー

SPAベースがその端末やブラウザプロファイルレベルで、えっと、ユニークに管理するなんかセッションIDみたいなものがあるんですか？

アシスタント

はい、Supabaseの認証機能では、JWTトークンのほかに、ユーザーがログインした際にサーバー側でセッションを管理し、それぞれのログインに対してユニークなセッションIDを発行—

ユーザー

で、ログインした際じゃ遅いんですよ。永続的に管理できなきゃダメなんですよ。

アシスタント

ログイン前であっても、永続的な識別子を管理する方法はあります。

まず、クライアントが初めてアクセスした際に、ブラウザのlocalStorageなどを利用して固有のIDを生成し、保存しておきます。

このIDをFCMの登録トークンと一緒にデータベースに保存しておけば、ログイン前であっても端末やブラウザを特定して管理することができますよ。

ユーザー

それ私がさっき言ったやつと同じじゃないかと思ったんですけど、違うんですね。えーっと、置き換えるトークをキーにするんじゃなくて、まずセッションが始まった時にGUID発行してローカルストレージに「私」という1つのオブジェクトIDを持っておくみたいな概念なんですね。

アシスタント

はい、まさにその通りです。初回アクセス時にUUIDを発行してローカルストレージに保存し、それをそのデバイスやブラウザプロファイルの「固有ID」としてデータベースで管理するという方法ですね。

この仕組みであれば、ログイン前であっても、あるいはログインユーザーが複数の端末や別々のプロファイルでアクセスしていても、正確に区別して通知トークンと紐づけることができます。

これこそが、求めているらっしゃる、確実に効率的な通知管理の基盤になりますよ。

ユーザー

よし、じゃあログインが完了したらUUIDを発行してローカルストレージに保存しておく。次回以降はそのローカルストレージの値を使い回すっていうことが1番いい設計ですね。

アシスタント

その通りです!その設計であれば、どの端末やブラウザからアクセスされても、一貫して同一の識別子を保持できるため、通知トークンを正確に管理しやすくなりますよ。

ユーザー

FCMのトークンIDを、え、Super StreamのEdge Functionsに対して、この、このIDを登録して下さいというメッセージを飛ばす、ま、いわゆるファンクションコールするための、い、きっかけというか、その契機っていうのがどのくらいの種類あるかっていうのをちょっと書き出して下さい。

アシスタント

SupabaseのEdge Functionsを呼び出すきっかけには、主に3つの種類が考えられます。

1つ目は、アプリケーションが初期化されたタイミングで、登録トークンを確実に保存または更新するためです。

2つ目は、FCM側でトークンが変更された際に自動的に実行されるイベントを検知した時です。

3つ目は、通知を送ろうとした際に、該当するトークンが無効であるというエラーを検知した時ですね。

4. 概念構造図 (Mermaid形式)

